

Project Proposal

Abstract

This project proposes the development of an AI-powered language learning application that provides personalized and adaptive learning experiences for users. Unlike traditional platforms with fixed lesson structures, the system dynamically adjusts content based on the learner's level, goals, and progress. By integrating Generative AI, the application will generate customized exercises, offer real-time feedback, and simulate interactive conversations. The goal is to help users learn more efficiently by focusing on their individual weaknesses and real-life communication needs, such as travel preparation or exam practice.

Main Functionality

The core functionality of the application is to deliver a **personalized language learning experience** through:

- Adaptive learning plans based on user level, goals, and deadlines
- AI-generated vocabulary, grammar, and conversation exercises
- Real-time and asynchronous feedback on user performance
- Interactive conversation practice with instant corrections
- Progress tracking and dynamic difficulty adjustment

The system continuously updates learning content based on user performance, ensuring efficient and targeted improvement.

Intended Users

The application is designed for a broad range of language learners, including:

- **Students** learning a foreign language in school or university
- **Self-learners** seeking flexible and personalized study options
- **Travelers** preparing for short-term practical communication needs
- **Professionals** improving language skills for work
- **Exam candidates** preparing for speaking or proficiency tests

The platform supports multiple languages, including German, English, French, Spanish, and Italian, making it accessible to a wide audience.

Meaningful Integration of GenAI

Generative AI serves as the **core engine** of the application rather than a secondary feature. It will be used to:

- Generate personalized exercises based on user weaknesses
- Provide clear, simple explanations for mistakes
- Adapt task difficulty dynamically in real time
- Simulate realistic conversations for speaking practice
- Analyze user input (text or speech) and provide targeted feedback
- RAG framework for specific usage (like interview preparing and training)

Application Scenario: Job Interview Preparation

The application will focus on helping users prepare for job interviews in a foreign language. The user enters their target role, current language level, interview date, and learning goal. Based on this information, the AI generates a personalized preparation plan with interview vocabulary, common questions, speaking practice, and feedback.

The main scenario is divided into smaller sub-scenarios:

1. Self-Introduction Practice

The user practices answers to questions like:

Tell me about yourself.

Why are you interested in this job?

The AI gives feedback on grammar, clarity, and formality.

2. Role-Specific Vocabulary

The AI generates vocabulary related to the user's target role, for example software engineer, data analyst, or project manager.

3. Mock Interview Practice

The AI simulates an interview by asking realistic questions. The user answers, and the AI gives feedback after each answer or at the end.

4. Feedback and Improvement

The system identifies weak areas such as grammar, vocabulary, fluency, or professional tone, then suggests exercises to improve them.

Tools and Technologies

Tools and Technologies

The system will be built using a combination of modern AI, backend, and frontend technologies to ensure scalability, responsiveness, and high-quality user interaction.

1. AI and Language Processing

- **Large Language Models (LLMs)** (e.g., OpenAI GPT models) for generating personalized exercises, explanations, and conversational responses
- **Retrieval-Augmented Generation (RAG)** using LlamaIndex and LangChain to incorporate structured learning materials (e.g., interview preparation, exam formats) into AI responses
- **Speech-to-Text APIs** (e.g., Google Speech-to-Text, OpenAI Whisper) for converting user speech into text
- **Text-to-Speech APIs** for generating natural-sounding AI responses during conversation practice

2. Backend Development

- **Frameworks** such as FastAPI or Node.js for building scalable APIs and handling application logic
- **Asynchronous task processing** (e.g., background jobs for generating learning plans and analyzing performance) to improve efficiency and responsiveness

3. Frontend Development

- **Web and mobile frameworks** like React or Flutter to create an interactive and user-friendly interface
- Real-time UI components for chat-based interaction and speaking practice

4. Data Storage and Management

- **Relational databases** such as PostgreSQL for structured user data (profiles, progress, plans)
- **NoSQL databases** like MongoDB for flexible storage of learning content and interaction logs